

Lab 2: C Libraries, Linking, and ELF Executable Structure

I. INTRODUCTION

When a programmer writes a C program that uses library functions like `printf()`, `getpid()`, and `time()`, compiles it, and runs it, a remarkable chain of events occurs. The compiler transforms human-readable C code into machine instructions. The linker resolves references to external libraries like `libc`. The operating system reads the resulting binary and maps it into virtual memory.

This lab answers three concrete questions:

- 1) How does a C program link to the Standard Library?
- 2) Where do C library headers and compiled code physically live on disk?
- 3) What is the difference between static and dynamic linking, and how can we inspect the resulting executables?

II. A PRACTICAL C PROGRAM: PROCESS INFORMATION

We begin with a C program that demonstrates real library usage by displaying process information. This program uses multiple functions from the C Standard Library and POSIX extensions:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>

int main(void) {
    // Get current process ID
    pid_t my_pid = getpid();
    pid_t my_ppid = getppid();

    // Get current time
    time_t current_time = time(NULL);
    struct tm *time_info =
        localtime(&current_time);

    // Format and display information
    printf("=== Process Information\n");
    printf("My Process ID (PID):  %d\n",
        my_pid);
    printf("Parent Process ID:   %d\n",
        my_ppid);
    printf("Current Time:         %s",
        asctime(time_info));
    printf("Executable Path:
        /proc/self/exe\n");

    return 0;
}
```

Save this as `procinfo.c` and compile it:

```
$ gcc procinfo.c -o procinfo
$ ./procinfo
=== Process Information ===
My Process ID (PID):  12345
Parent Process ID:    12344
Current Time:         Mon Sep 28 14:32:15
                        2026
Executable Path:      /proc/self/exe
```

This program uses several library functions:

- `getpid()` and `getppid()` from `<unistd.h>`
- `time()` and `localtime()` from `<time.h>`
- `printf()` and `asctime()` from `<stdio.h>`

All of these functions are implemented in the C Standard Library (`libc`). But where does the actual machine code for these functions come from?

III. THE C STANDARD LIBRARY: TWO HALVES

Every C program implicitly depends on the C Standard Library, commonly called `libc`. On Linux, the GNU implementation is `glibc`. This library is split into two distinct physical components that serve fundamentally different purposes.

A. Part 1: Header Files — The Construction Manual

Header files (`.h`) contain *declarations*: function prototypes, macro definitions, type definitions (`typedef`), and structure definitions. They tell the compiler *what* functions exist and *how to call them*, but they contain **no executable machine code**.

Analogy: A header file is like a construction manual.

It tells the architect (the compiler) the dimensions and interface of every component, but it does not contain the actual bricks and steel.

Physical location:

```
/usr/include/
```

We can verify this on any Linux system:

```
$ ls /usr/include/stdio.h
    /usr/include/unistd.h
/usr/include/stdio.h
/usr/include/unistd.h

$ grep "printf" /usr/include/stdio.h |
  head -2
extern int printf (const char *__restrict
    __format, ...);

$ grep "getpid" /usr/include/unistd.h |
  head -2
extern __pid_t getpid (void) __THROW;
```

When you write `#include <stdio.h>`, the preprocessor literally copies the contents of this file into your source code before compilation begins. The compiler now knows that `printf()` exists and what arguments it expects, but it does not yet know *where* the actual code lives.

B. Part 2: Precompiled Machine Code — The Actual Libraries

The actual compiled machine instructions for functions like `printf()`, `getpid()`, `time()`, and `malloc()` live in *library files*. These come in two forms:

- **Static libraries (.a):** Archives of object files. The linker copies the needed code directly into your executable.
- **Shared/Dynamic libraries (.so):** Standalone binaries that are loaded into memory at runtime by the dynamic linker. Multiple programs share a single copy in RAM.

Physical locations:

```
/lib/x86_64-linux-gnu/
/usr/lib/x86_64-linux-gnu/
/usr/lib/
/lib/
```

We can verify the presence of `libc`:

```
$ ls /lib/x86_64-linux-gnu/libc.so*
/lib/x86_64-linux-gnu/libc.so.6

$ ls /usr/lib/x86_64-linux-gnu/libc.a
/usr/lib/x86_64-linux-gnu/libc.a
```

C. Summary: The Two-Part Anatomy

TABLE I: Two Parts of the C Standard Library

Component	Contains	Location
Headers (.h)	Declarations, prototypes, macros	/usr/include/
Static lib (.a)	Precompiled object code (archive)	/usr/lib/
Shared lib (.so)	Precompiled position-independent code	/lib/

IV. STATIC VS. DYNAMIC LINKING

Linking is the process of resolving symbolic references (e.g., the name “`printf`”) to actual memory addresses. The timing and method of this resolution define two fundamentally different approaches.

A. Static Linking

In static linking, the linker (`ld`) copies the required machine code from `.a` archive files directly into the final executable at compile time.

```
$ gcc -static procinfo.c -o procinfo_static
$ ls -lh procinfo_static
-rwxr-xr-x 1 user user 850K ...
  procinfo_static
```

Characteristics:

- The executable is self-contained; it needs no external `.so` files to run.
- The binary is significantly larger (850KB vs 16KB) because it embeds all library code.

- Updating a library requires recompiling every program that uses it.
- Useful for embedded systems, containers, and environments where portability is critical.

B. Dynamic Linking

In dynamic linking, the executable contains only *references* to shared libraries. The actual code is loaded into memory at runtime by the *dynamic linker* (typically `/lib64/ld-linux-x86-64.so.2`).

```
$ gcc procinfo.c -o procinfo_dynamic
$ ls -lh procinfo_dynamic
-rwxr-xr-x 1 user user 16K ...
  procinfo_dynamic
```

Characteristics:

- The executable is small; library code is shared in RAM across all running programs.
- Updating `libc.so` instantly benefits every program that uses it.
- The program will fail to start if a required `.so` file is missing (the infamous “shared library not found” error).
- This is the default on virtually all modern Linux systems.

C. Why It Matters

Dynamic linking reduces memory consumption and disk usage across the entire system. However, it introduces runtime dependencies and potential version conflicts (“dependency hell”). Static linking eliminates these problems at the cost of larger binaries and no automatic security patching.

V. INSPECTING EXECUTABLES WITH `readelf`

On Linux, compiled executables use the **ELF** (Executable and Linkable Format) binary format. The `readelf` tool allows us to read the internal metadata of an ELF file without executing it.

A. Three Views of an Executable

An ELF binary can be understood through three complementary views:

- 1) **ELF Header** (`readelf -h`): Global metadata about the file—is it a valid executable, and for which CPU?
- 2) **Program Headers** (`readelf -l`): Instructions for the OS loader—which segments to map into memory and with what permissions.
- 3) **Dynamic Section** (`readelf -d`): Metadata for the dynamic linker—which shared libraries must be loaded.

B. View 1: The ELF Header (`readelf -h`)

```
$ readelf -h procinfo_dynamic
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 ...
  Class:   ELF64
  Data:    2's complement, little endian
  Type:    DYN (Shared object file)
  Machine: Advanced Micro Devices X86-64
  Entry point address: 0x1060
  ...
```

Key fields:

- **Magic:** The bytes 7f 45 4c 46 (“.ELF”) identify this as a valid ELF file.
- **Class:** ELF64 means 64-bit.
- **Machine:** Must match the host CPU. An ARM binary will not execute on x86_64.
- **Type:** EXEC (static) or DYN (position-independent, used for both shared libraries and modern dynamically-linked executables).

C. View 2: The Program Headers (`readelf -l`)

The Program Headers describe *segments*—contiguous regions of the file that the OS loader maps into virtual memory.

```
$ readelf -l procinfo_dynamic

Program Headers:
Type           Offset           VirtAddr           FileSiz MemSiz  Flags
PHDR           0x000040          0x...040
0x002d8 0x002d8 R
INTERP         0x000318          0x...318
0x0001c 0x0001c R
LOAD           0x000000          0x...000
0x005f4 0x005f4 R
LOAD           0x001000          0x...000
0x001a5 0x001a5 R E
LOAD           0x002000          0x...000
0x00120 0x00120 R
LOAD           0x002e10          0x...e10
0x00228 0x00230 RW
DYNAMIC        0x002e20          0x...e20
0x001d0 0x001d0 RW
GNU_STACK      0x000000          0x...000
0x00000 0x00000 RW
...
```

Critical entries to look for:

- **INTERP:** Specifies the path to the dynamic linker (e.g., `/lib64/ld-linux-x86-64.so.2`). The kernel reads this and loads the dynamic linker *before* the program’s `main()` runs.
- **LOAD:** Each LOAD segment corresponds to a memory region:
 - R only: typically `.rodata` (read-only constants).
 - R E: the `.text` segment (executable machine code).
 - RW: the `.data` and `.bss` segments (read-write global variables).
- **GNU_STACK:** Declares the stack’s permissions. RW means the stack is readable and writable but **not executable**—this is the secure default that prevents stack-based buffer overflow exploits. If this shows RWE, the stack is dangerously executable.

D. View 3: The Dynamic Section (`readelf -d`)

This section exists only in dynamically-linked executables. It tells the dynamic linker which shared libraries to load and where to find symbols.

```
$ readelf -d procinfo_dynamic

Dynamic section at offset 0x2e20 contains
28 entries:
Tag              Type              Name/Value
0x00000001 (NEEDED) Shared library:
    [libc.so.6]
0x0000000c (INIT)   0x1000
0x0000000d (FINI)   0x11a0
0x0000001d (RUNPATH) Library runpath:
    [/usr/lib]
...
```

Critical entries:

- **NEEDED:** Lists every shared library the program requires. The dynamic linker will search for and `mmap()` each of these into the process’s address space *before* `main()` begins. If any NEEDED library is missing, the program will refuse to start.

VI. TRACING DEPENDENCIES WITH `ldd`

While `readelf -d` shows the *names* of required libraries, `ldd` goes further: it actually invokes the dynamic linker to resolve each NEEDED entry to a *full filesystem path*.

```
$ ldd procinfo_dynamic
linux-vdso.so.1 (0x00007ffd...)
libc.so.6 =>
/lib/x86_64-linux-gnu/libc.so.6
(0x00007f...)
/lib64/ld-linux-x86-64.so.2
(0x00007f...)
```

What this tells us:

- `libc.so.6` is found at `/lib/x86_64-linux-gnu/libc.so.6`.
- `linux-vdso.so.1` is a virtual shared object injected by the kernel into every process. It provides fast system call entry points without a full context switch.
- `/lib64/ld-linux-x86-64.so.2` is the dynamic linker itself—the program that reads the INTERP and NEEDED entries and loads everything.

A. Static Binaries and `ldd`

If we run `ldd` on a statically-linked binary, the output confirms that no dynamic dependencies exist:

```
$ ldd procinfo_static
not a dynamic executable
```

This is because the static binary has no DYNAMIC section and no INTERP entry in its program headers.

VII. COMMAND REFERENCE SUMMARY

Table II consolidates the key tools, the specific fields to inspect, and what each field tells the operating system.

TABLE II: Command Reference: Inspecting ELF Executables

Command / Term	Look For	What It Tells the OS
<code>readelf -h <file></code>	Type, Machine	“Is this a valid executable, and does it match my CPU architecture?”
<code>readelf -l <file></code>	INTERP	“Which dynamic linker must run first to handle shared libraries?”
<code>readelf -l <file></code>	GNU_STACK	“Is this stack secure (RW) or dangerously executable (RWE)?”
<code>readelf -l <file></code>	LOAD (Flags)	“Which memory regions are code (R E), read-only data (R), or read-write data (RW)?”
<code>readelf -d <file></code>	NEEDED	“What external shared libraries must be mapped into memory before <code>main()</code> starts?”
<code>ldd <file></code>	Full output	“Where exactly on the filesystem are the required <code>.so</code> files located?”
<code>ldd <file></code>	“not a dynamic executable”	“This binary is statically linked and has no runtime library dependencies.”

VIII. CONCLUSION

This lab examined how a practical C program using multiple library functions (`printf`, `getpid`, `time`) links to the C Standard Library, which consists of header files in `/usr/include/` (the construction manual) and precompiled machine code in `/lib/` and `/usr/lib/` (the actual libraries).

We contrasted static linking, which produces self-contained but large binaries, with dynamic linking, which produces small executables that share library code in RAM at runtime. We used `readelf` and `ldd` to inspect the three critical views of an ELF executable: the ELF Header (validity and architecture), the Program Headers (memory layout and security), and the Dynamic Section (runtime dependencies).

IX. KNOWLEDGE CHECK QUESTIONS

- 1) What is the fundamental difference between a C header file and a shared library?
- 2) what is static and dynamic linking?
- 3) What is ELF Header? Write down the commands to check headers.
- 4) What does bookmark means? provide example.
- 5) Your are required to create a proper documentation of your finding. and push it into github.

REFERENCES

- [1] R. E. Bryant and D. R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed. Pearson, 2015.
- [2] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Addison-Wesley, 2013.
- [3] J. R. Levine, *Linkers and Loaders*. Morgan Kaufmann, 2000.
- [4] Tool Interface Standard (TIS), “Executable and Linking Format (ELF) Specification,” Version 1.2, 1995.